INSIDE STLC ACADEMY

QA LEADERSHIP ACADEMY

QA Metrics Catalogue

The metrics worth reporting, what each one actually tells you, and the vanity metrics to stop reporting today.

Purpose

A metrics catalogue is a curated menu of measures a QA leader can choose from, each defined once so it means the same thing to everyone. Its purpose is to move the conversation away from activity ("we wrote 400 tests") and towards outcomes ("fewer defects reach customers, and we recover faster when they do"). The catalogue also serves a second, quieter purpose: to give you the language to retire the vanity metrics that make QA look busy but say nothing about quality.

When to use it

- When you inherit a team reporting the wrong things — as at Northstar, where test-case count and bugs-found are reported and escaped defects are not.
- When setting up quality reporting for the first time and deciding what to measure.
- When an executive asks "how do we know quality is improving?" and you need a defensible answer.
- As the source list from which you build a Quality Health Scorecard or an Executive Quality Dashboard.

Instructions

1. Pick a small set — five to eight metrics — that map to outcomes leaders care about, not everything you can count.
2. For each metric, agree a single definition, an owner, and a cadence before you report it once.
3. Always pair a metric with context: a trend over time and a "what would make this better or worse" note. A number alone invites the wrong conclusion.
4. Read the caution column for every metric — each one can be gamed or misread, and knowing how is part of using it responsibly.
5. Retire the vanity metrics deliberately and explain why, so their disappearance is a decision, not an omission.

Worked example

A catalogue of meaningful QA and delivery metrics. The values are deliberately left out — this is the reference definition; you supply your own measured data over time.

Metric	Definition	What it tells you	Caution
Escaped defects	Defects found in production that testing should reasonably have caught, per release or per period.	The single clearest signal of testing effectiveness where it matters — after release.	Needs a fair "should reasonably have caught" rule, or it becomes a blame stick.
Production incidents	Count and severity of customer-affecting incidents attributable to a change.	Whether releases are getting safer or riskier over time.	Driven by many factors beyond QA; read as a system signal, not a QA scorecard.
Change failure rate	Proportion of releases that cause a failure needing a fix, rollback or hotfix.	How reliably the delivery system ships safe change — a core delivery-health measure.	Requires an agreed definition of "failure"; easy to under-count by relabelling incidents.
Rework	Effort spent redoing work due to defects, reopened tickets or failed acceptance.	The hidden cost of poor quality upstream; a strong ROI argument for earlier QA involvement.	Hard to measure precisely; use trend and proportion, not false precision.
Defect ageing	How long open defects have been open, banded by age and severity.	Whether defects are being resolved or quietly accumulating as debt.	Old low-severity defects can be fine; always band by severity before drawing conclusions.
Flaky-test rate	Proportion of automated tests that pass and fail without a code change.	How much you can trust your automation — and how much time it wastes.	A falling rate can mean flaky tests were deleted, not fixed; check alongside coverage.
Pipeline feedback time	Time from a commit to a trustworthy pass/fail result from the pipeline.	How fast the team learns it broke something; long feedback slows everything.	Fast but flaky feedback is worse than slow and reliable; pair with flaky-test rate.
Customer-reported defects	Defects raised by customers or support rather than found internally.	What your testing and monitoring are missing entirely, from the user's side.	Under-reported by users; a low count may mean low reporting, not high quality.
Risk coverage	Proportion of identified Critical/High product risks with adequate test coverage.	Whether effort is aimed at what matters, rather than at what is easy to test.	Only as honest as the risk assessment behind it; garbage risks give garbage coverage.
MTTD (mean time to detect)	Average time from a defect being introduced or an incident starting to it being detected.	How good your monitoring and alerting are at catching problems early.	An average hides outliers; the worst case often matters more than the mean.
MTTR (mean time to recover)	Average time from detection to service being restored.	Resilience — how quickly you limit the damage when something does go wrong.	Improving MTTR must not become an excuse to tolerate more incidents.

Vanity metrics to avoid — and why

Vanity metric	Why it misleads	Report this instead
Number of test cases	Measures volume of documentation, not coverage of risk. A thousand shallow cases beat by ten sharp ones.	Risk coverage of Critical/High risks.

Vanity metric	Why it misleads	Report this instead
Raw bug count (bugs found)	Rewards finding many trivial bugs and punishes a clean, well-built feature. More bugs is not more value.	Escaped defects and severity mix.
Pass rate without context	A 98% pass rate is meaningless if the 2% is the payment path, or if half the suite is flaky, or the plan was never finished.	Residual risk and what was not tested.
Automation count (tests automated)	Counts activity, not trust or value. Northstar has ~1,800 UI tests that are ~25% flaky and distrusted — a high number hiding a low signal.	Flaky-test rate and pipeline feedback time.
Tests executed	Rewards running tests, including low-value and duplicate ones, regardless of what they protect.	Change failure rate and escaped defects.

Blank reusable version

Metric	Definition	What it tells you	Caution	Owner	Cadence

Use the Metric Definition Template to specify each chosen metric in full before it enters regular reporting.

Common mistakes

COMMON MISTAKES

Reporting activity metrics (test-case count, bugs found, tests executed) because they are easy to count and make QA look busy; tracking a dozen metrics nobody acts on; reporting a single number with no trend and no context; setting targets on a metric before understanding how it can be gamed; and using escaped defects or bug counts to blame individuals, which teaches people to hide problems rather than surface them.

- Comparing metrics across teams with different definitions, so the comparison is meaningless.
- Optimising one metric in isolation — driving down flaky-test rate by deleting tests, for instance.

How to interpret the results

- Read metrics in balanced pairs, never alone: escaped defects with risk coverage, MTTR with incident count, flaky-test rate with pipeline feedback time. A pair resists gaming; a single number invites it.
- Trend beats snapshot. "Escaped defects are falling three periods running" tells you more than any single figure.
- A metric that no decision depends on should be dropped. If nothing changes whether it is red or green, it is decoration.
- When a metric looks great in isolation, ask what it might be hiding — a superb pass rate over a distrusted, flaky suite is a warning, not a comfort.
- Use metrics to ask better questions, not to hand out verdicts. Their job is to point you at where to look.

INSIDE STLC PRO TIP

> The fastest way to earn a data-driven leader's respect is to walk in and retire your own vanity metrics before anyone asks. Telling a VP Engineering "I have stopped reporting test-case count because it measures typing, not quality — here is escaped-defect trend instead" signals judgement that no dashboard ever will.